

گزارش کار

ساختار و زبان کامپیوتر

بهمن ماه ۱۴۰۳

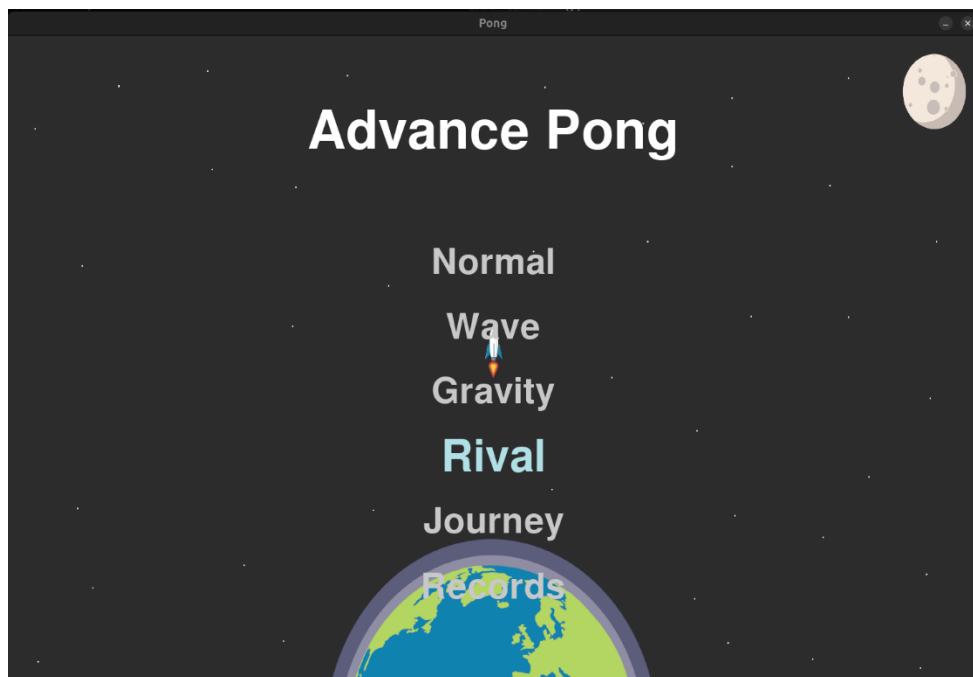
—

Professor Jahangir

—

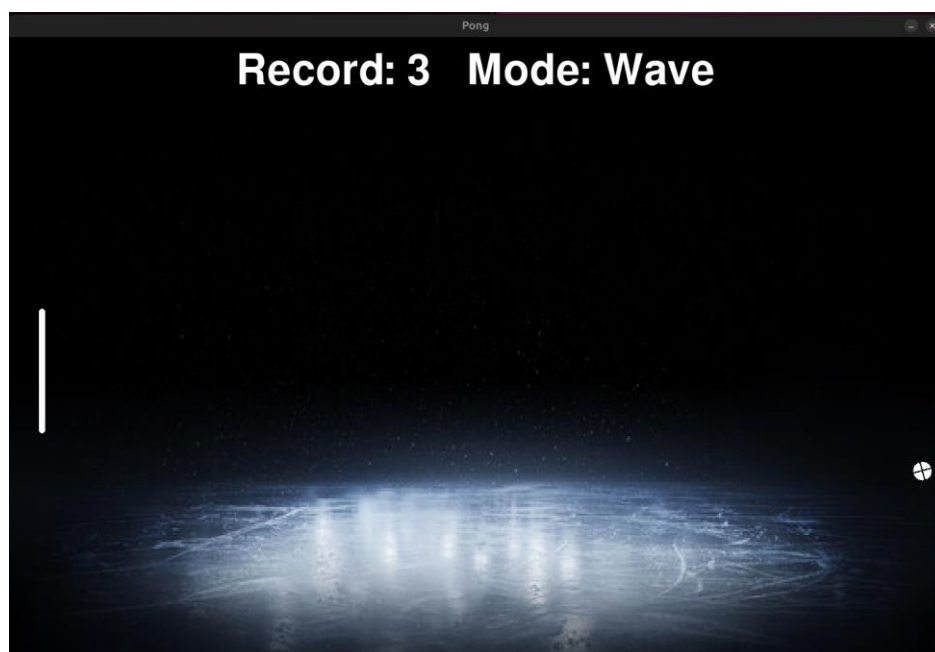
توضیحات بخش های مختلف بازی:

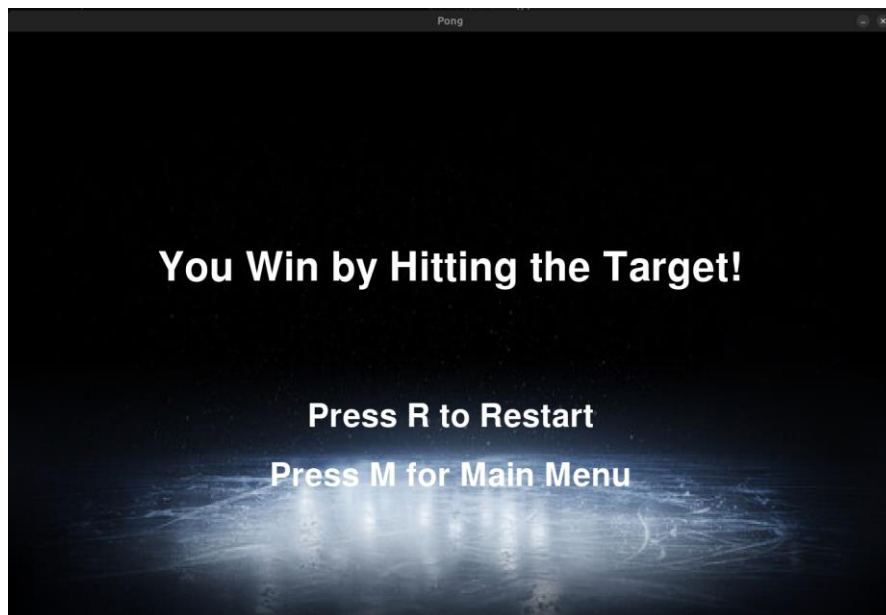
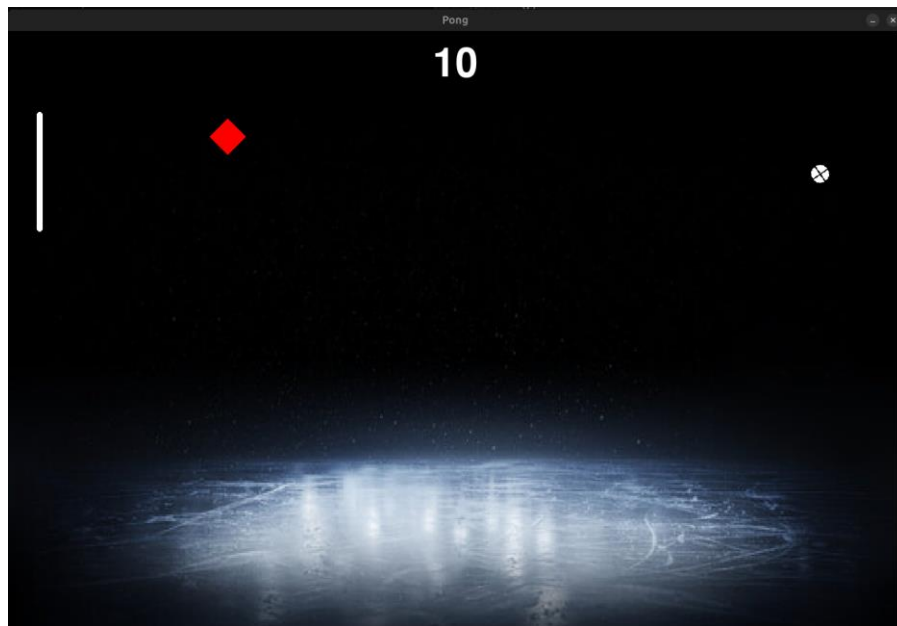
۱- منو اصلی:



همانطور که از منوی اصلی بازی مشخص است. بازی ۵ mode مختلف دارد که توضیحات هر کدام به ترتیب در پایین تر آمده است:

۱-۱- عادی (Normal): در این بخش توپ به صورت عادی حرکت می کند. البته به دلیل یکنواخت نشدن بازی و نیفتادن بازی داخل حلقه ای که نشود از آن خارج شد با توجه به حرکت ثابت توپ این رو در نظر گرفتم که زاویه بازتاب توپ وابسته به محل برخورد توپ به راکت باشد. که اگر به بالای راکت برخورد کرد به سمت بالا منعکس بشود و در صورت برخورد به پایین راکت بالعکس. همچنین در ابتدای شروع هر بازی با هر mode شمارش معکوسی از ۳ پخش می شود که بازیکن آماده بازی بشود. و همچنین برای معنادار بودن برد و باخت در این مدل و همچنین مدل Wave و Gravity که جلوتر به آنها خواهیم پرداخت در نظر گرفته شده است که با رسیدن بازیکن به ۱۰ امتیاز یک الماس قرمز به روی صفحه بیاید تا بازیکن تلاش کند که توپ را به آن برخورد بدهد.





```
# ----- Ball Collision with Top/Bottom -----
if ball.top < 0:
    ball.top = 0
    ball_velocity[1] = -ball_velocity[1]
    collision_sound.play()
if ball.bottom > HEIGHT:
    ball.bottom = HEIGHT
    ball_velocity[1] = -ball_velocity[1]
    collision_sound.play()

# ----- Ball Collision with Paddle -----
if ball.colliderect(paddle):
    relative_intersect = (paddle.centery - ball.centery) / (PADDLE_HEIGHT / 2)
    ball_velocity[1] = -relative_intersect * 10
    ball_velocity[0] = -ball_velocity[0]
    ball.x += 17
    collision_sound.play()
    if game_mode == "journey":
        journey_step_points += 1
        journey_total_points += 1
        if journey_step_points >= 3:
            journey_step_points = 0
            journey_step_index = (journey_step_index + 1) % 3
            if journey_step_index == 0:
                journey_speed_multiplier += 0.2
    else:
        if game_mode != "rival" and not target_active:
            score += 1
        elif game_mode == "rival":
            score += 1
```

بخش قرار داده شده از کد بازی در بالا برای اجرای برخورد نسبی توپ با راکت و دیواره های بالا و پایین است که اگر با برخورد داشت با سقف یا کف بازتاب می شود و همچنین اگر با راکت برخورد داشته باشد سرعت عمودی توپ متغیر خواهد بود و به اصطلاح برخورد الاستیک نخواهد بود. همچنین مقدار X توپ به اندازه عرض پدل افزایش می یابد تا اگر به گوشه های راکت برخورد داشت توپ گیر نکند در راکت.

```
# ----- Right-Side Collisions -----
if game_mode in ("normal", "wave", "Gravity") or game_mode == "journey":
    if ball.colliderect(wall):
        ball_velocity[0] = -ball_velocity[0]
        collision_sound.play()
```

این بخش هم برای برخورد توپ به دیواره ی سمت راست قرار داده شده تا بازتاب بشود.

```
def update_ball_x(ball_x: float, ball_velocity_x: float) -> float:
    return ball_x + ball_velocity_x

def update_ball_y(ball_y: float, ball_velocity_y: float) -> float:
    return ball_y + ball_velocity_y
```

این هم نحوه حرکت توپ در حالت عادی

```
# ----- Countdown Before Game Starts -----
if not game_started:
    if countdown_start_time is None:
        countdown_start_time = pygame.time.get_ticks()
    elapsed = (pygame.time.get_ticks() - countdown_start_time) / 1000
    countdown_value = 3 - int(elapsed)
    screen.blit(game_bg, (0, 0))
    draw_capsule_paddle(screen, WHITE, paddle)
    if game_mode == "rival":
        pygame.draw.rect(screen, WHITE, goaler)
    else:
        pygame.draw.rect(screen, WHITE, wall)
    if countdown_value > 0:
        draw_countdown(countdown_value)
        pygame.display.flip()
        clock.tick(60)
        continue
    else:
        game_started = True
```

شمارش معکوس در ابتدای بازی

```
# ----- Target Activation for Non-rival Modes (ignored in Journey mode) -----
if game_mode not in ["rival", "journey"] and score >= 10 and not target_active:
    target_active = True
    target_center = (random.randint(100, WIDTH - 100), random.randint(100, HEIGHT - 100))
    target_size = 60
    target_points = [
        (target_center[0], target_center[1] - target_size // 2),
        (target_center[0] + target_size // 2, target_center[1]),
        (target_center[0], target_center[1] + target_size // 2),
        (target_center[0] - target_size // 2, target_center[1])
    ]
    target_rect = pygame.Rect(target_center[0] - target_size // 2,
                              target_center[1] - target_size // 2,
                              target_size, target_size)
```

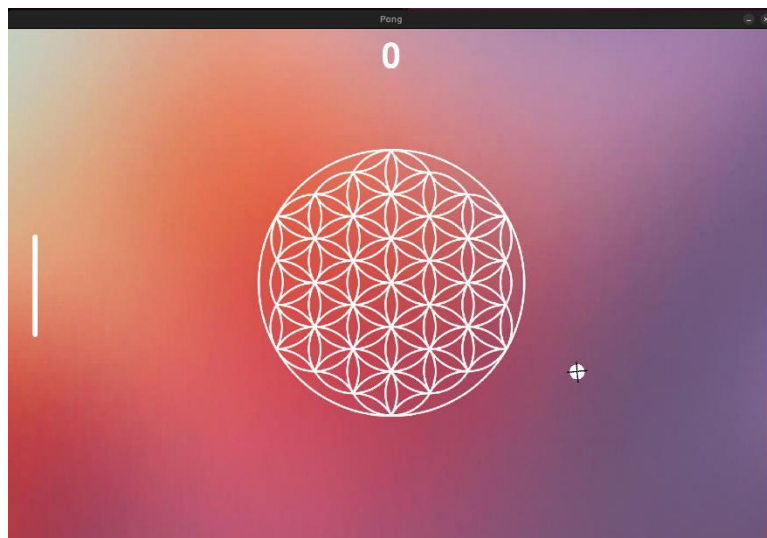
گذاشتن الماس در مکانی اتفاقی از صفحه

۱-۲- موجی (wave): در این حالت توپ حرکت عادی خود را دارد اما حول مسیر اصلی حرکتی سینوسی دارد که دریافت آن را سخت تر می کند. محاسبه مقدار تابع سینوس حرکت موجی با توجه به جایگاه توپ.

```
def calculate_wave_offset(x: float) -> float:
    return math.sin(x * wave_frequency) * wave_amplitude

elif current_underlying == "wave":
    ball.x = int(update_ball_x(ball.x, ball_velocity[0]))
    ball.y = int(update_ball_y(ball.y, ball_velocity[1]))
    wave_offset = calculate_wave_offset(ball.x)
    ball.y = int(ball.y + (wave_offset - calculate_wave_offset(ball.x - ball_velocity[0])))
```

حرکت توپ در حالت موجی که مقدار Y با توجه به تابع سینوس عوض می شود.



صفحه بازی در حالت موجی

۱-۳- جاذبه (Gravity): در این حالت انگار که جاذبه ای با معادله حرکت شتابدار ($ax^2 + bx + c$) حرکت می کند که برای اجرای آن مقدار سرعت عمودی توپ را با مقدار جاذبه جمع می کند.

```
def apply_gravity(ball_velocity_y: float) -> float:
    return ball_velocity_y + gravity
```

نحوه اجرا در کد بازی

البته برای میرا نشدن توپ در این حالت وقتی توپ در حین بالا آمدن است سرعت آن افزایش می یابد.

```
elif game_mode == "Gravity":
    ball.x = int(update_ball_x(ball.x, ball_velocity[0]))
    ball.y = int(update_ball_y(ball.y, ball_velocity[1]))
    ball_velocity[1] = apply_gravity(ball_velocity[1])
```

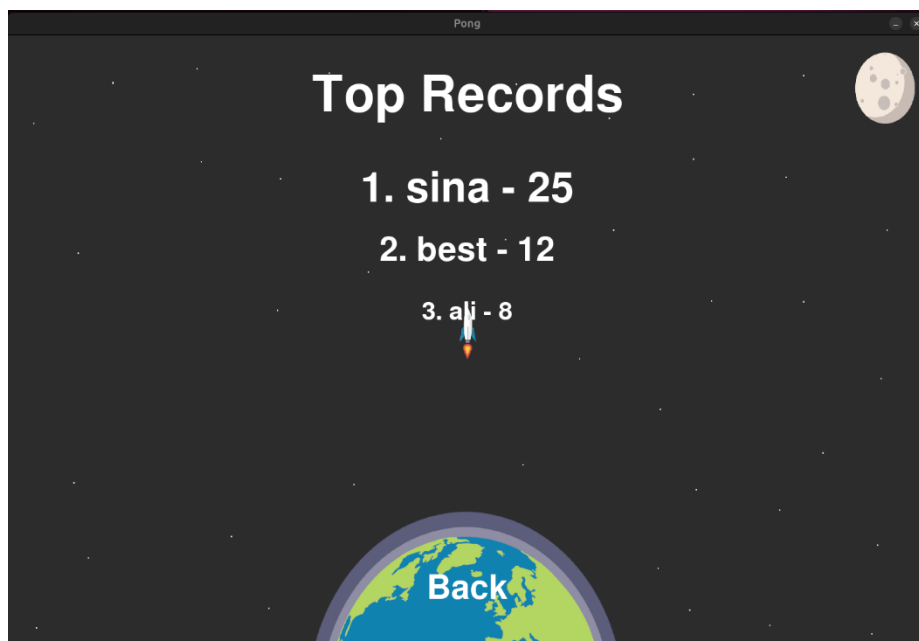

۱-۴- حالت ماجراجویی (journey): در این حالت هر سه امتیازی که گرفته بشود حالت بازی بین سه حالتی که پیشتر گفته شد عوض می شود و سرعت حرکت توپ هم در راستای محور X افزایش می یابد. این حالت بازی رکوردی می باشد و رکورد سه نفر برتر ذخیره می شود. که می توان در منوی record تاریخچه رکوردهای برتر را مشاهده کرد.

```
if game_mode == "journey":
    current_underlying = journey_modes[journey_step_index]
    if current_underlying == "normal":
        ball.x = int(update_ball_x(ball.x, ball_velocity[0]))
        ball.y = int(update_ball_y(ball.y, ball_velocity[1]))
    elif current_underlying == "wave":
        ball.x = int(update_ball_x(ball.x, ball_velocity[0]))
        ball.y = int(update_ball_y(ball.y, ball_velocity[1]))
        wave_offset = calculate_wave_offset(ball.x)
        ball.y = int(ball.y + (wave_offset - calculate_wave_offset(ball.x - ball_velocity[0])))
    elif current_underlying == "Gravity":
        ball.x = int(update_ball_x(ball.x, ball_velocity[0]))
        ball.y = int(update_ball_y(ball.y, ball_velocity[1]))
        ball_velocity[1] = apply_gravity(ball_velocity[1])
```

حرکت توپ در حالت رقابت.

```
# ----- Journey Mode Record Entry -----
if game_mode == "journey" and game_over and not record_entered:
    qualifies = False
    if len(records_list) < 3:
        qualifies = True
    else:
        lowest_score = records_list[-1][1]
        if journey_total_points > lowest_score:
            qualifies = True
    if qualifies:
        player_name = get_player_name()
        records_list.append((player_name, journey_total_points))
        records_list.sort(key=lambda x: x[1], reverse=True)
        records_list = records_list[:3]
        save_records(records_list)
    record_entered = True
```

تعیین ثبت شدن یا نشدن رکورد بازیکن.

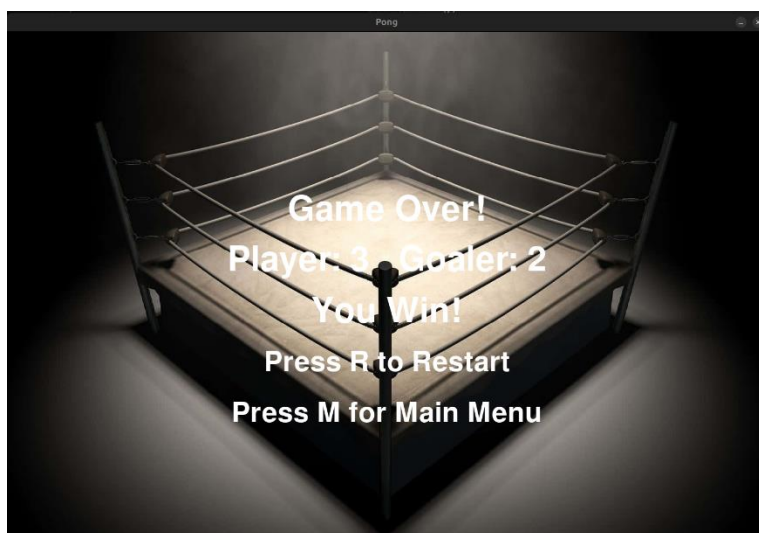


منوی رکوردها

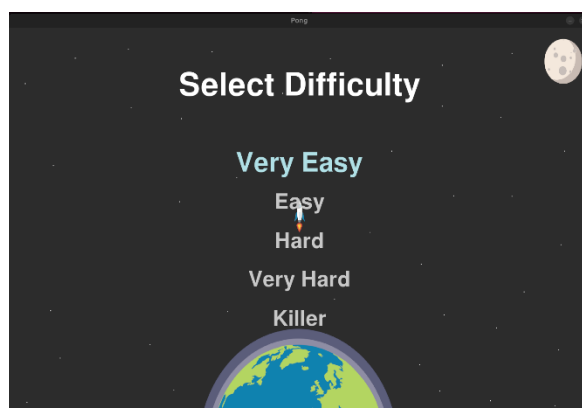
۵-۱- حالت رقابت (rival): در این حالت بازیکن یک بازی ۳ گله را با هوش مصنوعی انجام می‌دهد. همچنین در این بازی می‌توان با فشار دادن دکمه‌های H و L روی کیبورد خود سرعت حرکت راکت متعلق به هوش مصنوعی را کاهش یا افزایش دهد. که جذابیت بازی را تغییر می‌دهد. همچنین در ابتدای تمام این mode های بازی می‌توان درجه سختی را تعیین کرد که در ۵ سطح تعیین کرد که منجر به تغییر سرعت حرکت توپ می‌شود.



صفحه حالت رقابت.



صفحه تعیین برد و باخت.



منوی تعیین درجه سختی.

```

elif game_mode == "rival":
    if ball.centery > goaler.centery:
        goaler.y += int(goaler_speed)
    elif ball.centery < goaler.centery:
        goaler.y -= int(goaler_speed)
    if goaler.top < 0:
        goaler.top = 0
    if goaler.bottom > HEIGHT:
        goaler.bottom = HEIGHT
    if ball.colliderect(goaler):
        ball_velocity[0] = -abs(ball_velocity[0])
        ball.x = goaler.x - ball.width
        collision_sound.play()
    if ball.right < 0:
        goaler_score += 1
        if goaler_score < WIN_SCORE:
            reset_ball(difficulty)
        else:
            game_over = True
    elif ball.left > WIDTH:
        player_score += 1
        if player_score < WIN_SCORE:
            reset_ball(difficulty)
        else:
            game_over = True
    if player_score >= WIN_SCORE or goaler_score >= WIN_SCORE:
        game_over = True

```

حرکت رقیب به سمت توپ و برخورد توپ با راکت رقیب.

۲- بهینه سازی با استفاده از زبان assembly x86 : برای افزایش سرعت پردازش هر فریم در بازی برای محاسبه حرکت توپ از توابع assembly به جای پایتون بهره می‌بریم. که توابع جایگزین به شکل زیر است:

```

calculate_wave_offset:
    ; float calculate_wave_offset(float x)
    ; read input from xmm0
    mulss xmm0, dword [rel wave_frequency] ; x cross in wave frequency
    cvtss2sd xmm0, xmm0                     ; make it double to use it in fsin
    movq [rsp-8], xmm0                      ; save in memmory
    fld qword [rsp-8]
    fsin                                    ; calculate sin with fsin
    fstp qword [rsp-8]                     ; save in memmory
    movq xmm0, [rsp-8]                     ; load answer in xmm0
    cvtsd2ss xmm0, xmm0                    ; return it to float
    mulss xmm0, dword [rel wave_amplitude] ; cross in wave amplitude
    ret

apply_gravity:
    ; void apply_gravity(float* ball_velocity_y)
    ; read input from rdi
    movss xmm0, dword [rdi]                ; load ball velocity in xmm0
    addss xmm0, dword [rel gravity]         ; add to gravity
    movss dword [rdi], xmm0                ; save in ball velocity
    ret

update_ball_x:
    ; float update_ball_x(float ball_x, float ball_velocity_x)
    ; read ball_x in xmm0 and ball_velocity_x in xmm1
    addss xmm0, xmm1 ; ball_x + ball_velocity_x
    ret

update_ball_y:
    ; float update_ball_y(float ball_y, float ball_velocity_y)
    ; read ball_y in xmm0 and ball_velocity_y in xmm1
    addss xmm0, xmm1 ; ball_y + ball_velocity_y
    ret

;command to make a python library from assembly file:
;nasm -felf64 calculations.asm -o calculations.o && gcc -shared -o libcalculations.so calculations.o

```

این جایگذاری تقریباً باعث بهبود سرعت میانگین پردازش هر فریم به اندازه ۵۰ درصد شد که نتیجه تست در زیر آمده است:

```

~/De/final project/combined > python3 combinedVersion.py took 19s
<frozen importlib._bootstrap>:488: RuntimeWarning: Your system is avx2 capable b
ut pygame was not built with support for it. The performance of some of your bli
ts could be adversely affected. Consider enabling compile time detection with en
vironment variables like PYGAME_DETECT_AVX2=1 if you are compiling without cross
compilation.
pygame 2.5.2 (SDL 2.30.0, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
average: 0.010768083146836091 seconds for each frame

```



```
~/De/f/onlypython > python3 only\python.py took 23s
<frozen importlib._bootstrap>:488: RuntimeWarning: Your system is avx2 capable but pygame was not built with support for it. The performance of some of your blits could be adversely affected. Consider enabling compile time detection with environment variables like PYGAME_DETECT_AVX2=1 if you are compiling without cross compilation.
pygame 2.5.2 (SDL 2.30.0, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
average: 0.01825970710885529 seconds for each frame
```

حالت عادی

این تست به این گونه گرفته میشود که زمان کل اجرای بازی را تقسیم بر تعداد فریم‌های پردازش شده می‌کنیم (به این صورت که هربار وارد حلقه اجرای بازی می‌شویم تعداد فریم‌های شمرده شده را یک واحد افزایش می‌دهیم)

۳- برای لینک کردن پایتون به اسمبلی با استفاده از کتابخانه `ctypes` و همچنین خود `gcc` کد اسمبلی را به یک کتابخانه تبدیل می‌کنیم و در ابتدای فایل پایتون این کتابخانه ایجاد شده را به کد پایتون خود لینک می‌کنیم.

```
import ctypes

# Load the assembly library
lib = ctypes.CDLL('./libcalculations.so')

# Define function prototypes for assembly functions
calculate_wave_offset_asm = lib.calculate_wave_offset
calculate_wave_offset_asm.argtypes = [ctypes.c_float]
calculate_wave_offset_asm.restype = ctypes.c_float

apply_gravity_asm = lib.apply_gravity
apply_gravity_asm.argtypes = [ctypes.POINTER(ctypes.c_float)]
apply_gravity_asm.restype = None

update_ball_x_asm = lib.update_ball_x
update_ball_x_asm.argtypes = [ctypes.c_float, ctypes.c_float]
update_ball_x_asm.restype = ctypes.c_float

update_ball_y_asm = lib.update_ball_y
update_ball_y_asm.argtypes = [ctypes.c_float, ctypes.c_float]
update_ball_y_asm.restype = ctypes.c_float

# Replace Python functions with assembly versions
def calculate_wave_offset(x: float) -> float:
    return calculate_wave_offset_asm(x)

def apply_gravity(ball_velocity_y: float) -> float:
    y = ctypes.c_float(ball_velocity_y)
    apply_gravity_asm(ctypes.byref(y))
    return y.value

def update_ball_x(ball_x: float, ball_velocity_x: float) -> float:
    return update_ball_x_asm(ball_x, ball_velocity_x)

def update_ball_y(ball_y: float, ball_velocity_y: float) -> float:
    return update_ball_y_asm(ball_y, ball_velocity_y)
```

```
def reset_ball_journey():
    ball.x = WIDTH // 2 - BALL_SIZE // 2
    ball.y = HEIGHT // 2 - BALL_SIZE // 2
    base_speed_x = 7
    base_speed_y = 6
    effective_speed_x = max(1, int(base_speed_x * journey_speed_multiplier))
    effective_speed_y = max(1, int(base_speed_y * journey_speed_multiplier))
    ball_velocity[0] = random.choice([-effective_speed_x, effective_speed_x])
    ball_velocity[1] = random.choice([-effective_speed_y, effective_speed_y])
    global game_started, countdown_start_time
    game_started = False
    countdown_start_time = None
```

ریست کردن توپ در حالت ماجراجویی

```
# ----- Update Ball Rotation -----
ball_rotation_angle = (ball_rotation_angle + 5) % 360
rotated_ball_image = pygame.transform.rotate(ball_image, ball_rotation_angle)
ball_rect = rotated_ball_image.get_rect(center=ball.center)
```

چرخش توپ.

```
# ----- Player Paddle Movement -----
keys = pygame.key.get_pressed()
if keys[pygame.K_UP] and paddle.top > 0:
    paddle.y -= 7
if keys[pygame.K_DOWN] and paddle.bottom < HEIGHT:
    paddle.y += 7
if game_mode == "rival":
    if keys[pygame.K_h]:
        goaler_speed += 0.2
    if keys[pygame.K_l]:
        goaler_speed = max(1, goaler_speed - 0.2)
```

حرکت راکت و تنظیم سرعت حرکت راکت حریف در حالت رقابت.